

## TAKE-HOME BRIEF

# Technical Task

## Full Stack Engineer (F/M/D)

Barcelona · React, TypeScript, Python, Django

| FORMAT                 | TIME       | DATA          | RETURN WITHIN |
|------------------------|------------|---------------|---------------|
| Take-home, then a call | 8–10 hours | Overture Maps | One week      |

# Make a district legible

UrbView turns geospatial data into something a city planner can act on: a map of an area, and beside it the numbers that say what is there. Build that. Pick a city, take a district of it from Overture Maps, and give us a map with an interactive KPI dashboard wired to it.

Which KPIs is your call, and the choice is most of what we are assessing. Things Overture can answer: how much street length sits within a short walk of a lamp, how mixed the land use is block by block, how far the average building is from a transit stop, what share of the network is walkable, how many junctions per km<sup>2</sup>, how much of the area is water or green. Four or five you can defend beat a dozen you cannot.

The point is not the algorithm. It is whether the numbers you publish mean something, and whether someone else can see why.

## Four deliverables

### 1 Backend — Python, Django

An endpoint that takes an area — a GeoJSON polygon, a bbox, or a district id — and returns the KPIs plus the geometry behind them. REST or GraphQL, your choice; we run both. If you pick GraphQL, generate your client types from the schema rather than hand-writing them, because that is how we work.

Storage is your call too. We read Parquet with DuckDB rather than keeping geometry in Postgres, and either that or PostGIS is a good answer here. What we look at is that the clipping and the aggregation happen in the query engine, not in a Python loop over features.

### 2 Frontend — React, TypeScript, MapLibre GL

One page: the map and the dashboard, sharing one state. This is the deliverable we will spend the most time in, so:

- **Draw an area, and the dashboard follows it.** The core interaction, not a nice-to-have: draw a polygon anywhere on the map and every KPI recomputes for what falls inside it alone. Clearing the drawing returns to the whole district.
- At least one chart, and at least one KPI that is more than a count. A share, a density, a distance.
- It reads both ways. Click a category in the dashboard and the matching features highlight on the map; click a feature on the map and the dashboard says what it contributes.
- Loading, empty and error states exist. A drawn area with no lamps in it is a real case, not a crash.

### 3 NOTES.md — the reference

One table: each KPI, what it measures, how it is computed, its thresholds, and where each threshold came from. Plus a line per KPI on what it does **not** claim. A table, not an essay — the argument belongs in the walkthrough.

#### 4 A walkthrough — PDF, 8–12 slides

The deliverable that shows rather than tells, and the one we read first. It should carry:

- Your dashboard reading a real place, in screenshots. Pick a drawn area you find interesting, or the district whole, and say what the numbers actually tell you about it.
- A diagram of how the pieces fit: extraction, storage, query, client.
- The decisions you made and the ones you rejected. Why these KPIs, why this threshold, why this component boundary.
- What you would do with another week, and what you would throw away.
- What breaks at scale: what you would change if the area were ten times larger, or if the computation took ninety seconds instead of one.

Slides, Figma, Keynote, a Markdown deck — whatever you like, exported to PDF so it opens anywhere. We work through it with you in the review call.

### The response shape we have in mind

Close to our own, and a reasonable place to start. Deviate where you have a reason.

```
{
  "area": { "name": "Eixample", "km2": 7.46 },
  "kpis": [
    {
      "key": "lit_street_share",
      "label": "Street length within 25 m of a lamp",
      "value": 61.4,
      "unit": "%",
      "band": "Partly covered",
      "definition": "One sentence: what this measures, and what it does not."
    }
  ],
  "insights": [
    "A plain-language sentence generated from the data, not hardcoded."
  ],
  "legend": {
    "type": "categorical",
    "items": [{ "label": "Unlit", "color": "#b91c1c" }]
  },
  "layers": { "vectors": [ /* GeoJSON FeatureCollection(s) */ ] }
}
```

## Overture, and nothing else

One source. Pick a city, cut a district of roughly 2–10 km<sup>2</sup>, and pull it from [overturemaps.org](https://overturemaps.org) — their CLI, DuckDB against the S3 parquet, however you like.

No second dataset, and no OpenStreetMap to fill a gap: if Overture does not carry it, that absence is part of what you are reporting on. You start from nothing of ours — no data, no repository, no access — so the extraction is part of the work. Commit the extraction script, not the data.

| OVERTURE THEME               | WE CALL IT                                | TYPICALLY CARRIES                                                                                         |
|------------------------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| transportation               | <code>transportation</code>               | Street and path segments, connectors, road class, access restrictions                                     |
| places                       | <code>places</code>                       | Shops, services, anything staffed                                                                         |
| buildings                    | <code>buildings</code>                    | Footprints, heights, subtypes                                                                             |
| base — land, land use, water | <code>landuse</code> , <code>water</code> | Parks, blocks, rivers                                                                                     |
| base — infrastructure        | <code>street_lamps</code>                 | Street furniture. Overture models a lamp as <code>subtype=utility</code> , <code>class=street_lamp</code> |
| divisions                    | <code>boundaries</code>                   | District and city limits                                                                                  |

## Say where the number came from

**Every threshold in your KPIs has to come from somewhere we can check.**

If a standard, a paper, or a municipal guideline says 400 m is a reasonable walk to a bus stop, cite it. If you derived a cut from the data, show the method. If you picked a number because it looked right, write that down in `NOTES.md` — that is an acceptable answer, and a dishonest citation is not.

The second half of the rule is the one people forget: say what the number does not mean. A low reading on your dashboard should not be readable as “this place is dangerous” unless you can defend that, and you almost certainly cannot.

### No socioeconomic data. Not any of it.

Not income, wealth, rent or house prices. Not education, unemployment or benefits. Not deprivation indices of any kind. Not nationality, ethnicity or migration background. Not age structure or household composition. Not health, insurance or lending data.

Overture carries none of it, so the one-source rule mostly enforces this for you. We state it anyway because the tempting move is to reach outside for a denominator or a control variable. If a KPI needs a denominator, use street length, area, or building count, and say in `NOTES.md` which you used and why.

The reason is not squeamishness. A finding has to point at something a city can act on: a planner can add a lamp, widen a footway or open a frontage, but nobody pulls the lever marked “median income”. A number built on who lives somewhere also stops being a statement about the place and becomes one about the people in it. And it launders correlation into causation – a deprivation index correlates with recorded crime in most cities, so a score fed by one looks excellent while having learned nothing about the street.

## SCOPE & ASSESSMENT

# What good looks like

Eight to ten hours, the walkthrough included. We would rather see fewer KPIs done well, with a README that says what you cut and why, than everything rushed. If something has to give, cut KPIs – not the drawing, not the tests, not the walkthrough.

## What we need, and what we’d love

**Must have.** The four deliverables above, and:

- `docker compose up`, then one documented command to load the data.
- At least one test that pins a KPI’s computation – not that the endpoint returns 200 – and one frontend test.
- Fifteen minutes from a clean clone to a running app. If we cannot get there, we cannot review it.

**Extra.** Anything you build past that, we will read and count in your favour – auth, a job queue, a worker, deployment, more KPIs, a design system, a better-looking map. There is no ceiling on what you add. There is also no penalty for adding none of it: the must-haves are what we compare submissions on, so get those right first. One area at a time, computed synchronously, is fine. A hardcoded city is fine.

## How we assess it

Published so you can spend your hours where they count.

| DIMENSION                  | WEIGHT | WHAT WE LOOK FOR                                                                                                                                                                                        |
|----------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend and dashboard     | 25%    | Map and dashboard are one state, not two. The drawn area genuinely drives the numbers. Interaction reads both ways. Typed against the response, generated rather than hand-written if you chose GraphQL |
| Judgment about the numbers | 20%    | Every threshold traced to a source, or an honest “I chose this”. Each KPI says what it does not claim                                                                                                   |
| Backend and data access    | 20%    | Clipping and aggregation pushed into the query engine rather than a Python loop over features. Sensible indexing or partitioning. Migrations that run. No N+1                                           |
| The walkthrough            | 15%    | Shows the product reading a real place and says what it found. Explains decisions and rejected alternatives rather than narrating features. Knows what would break at ten times the size                |
| Tests and runnability      | 12%    | A test that would fail if a KPI’s computation changed. Clean clone to running app in under fifteen minutes                                                                                              |
| Docs and data extraction   | 8%     | <b>NOTES.md</b> is a usable reference table. The Overture pull scripted and committed, the data not                                                                                                     |

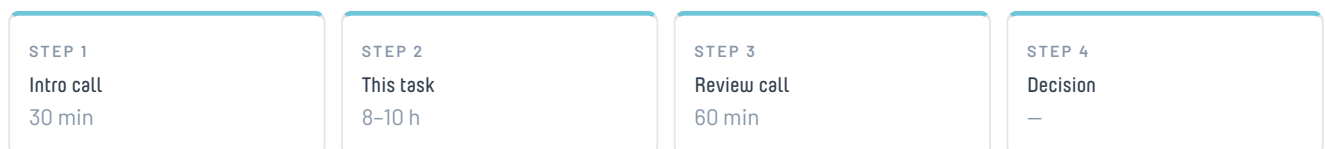
Extras sit on top of this table, never against it.

## Submitting

A Git repository with real commits, not one “initial commit” with everything in it. A README with setup, what you built, what you cut, and anything you want us to look at.

## PROCESS

# What happens next



The whole loop runs inside two weeks. You have a week to return the task; if you need more, ask and you will get it.

## The review call — one hour

We will have read your walkthrough before it starts. Around thirty minutes working through it with you — what you built and why we should believe the numbers — then fifteen minutes where we ask for a change and you make it while we watch, with whatever tools you normally use. It might be adding a KPI, making a chart click

filter the map, or working out why the endpoint takes nine seconds on a 10 km<sup>2</sup> polygon. We are watching how you decide, not how fast you type.

The last fifteen minutes are a conversation about SaaS billing with Stripe — webhooks, subscriptions, keeping a database in step with a payment provider. Nothing to prepare and nothing to build; we just talk through a couple of real situations.

## On AI

Use whatever tools you normally use, coding assistants included — we use them every day. What we ask is that you understand what is in your repository and can reason about why it is that way: why this threshold, why this component boundary, why this query and not the obvious one. The review call is where that comes up.

## What we do not ask for

No algorithm puzzles, no timed tests, no take-home that happens to be a ticket from our backlog, and no request to design our roadmap for free. The repository stays yours, and we will not use it in the product.

**If anything here is ambiguous, make a decision, write down why, and carry on. That is the job.**

---

Questions at any point — ask. We would rather answer than have you guess.